
«java.rmi»
Remote
Identity
remoteAskForChat(author: String, chatID: UUID): void
remoteApproveBackChat(chatID: UUID): void
remoteRefuseChat(chatID: UUID): void
remoteSendMessageInChat(chatID: UUID, safeMessage: SafeMessage): void
remoteSendBroadcastMessage(signedMessage: SignedMessage): void
remoteCloseChat(chatID: UUID): void
remoteAcknowledgeMessage(chatId: UUID, messageId: UUID): void
SafeMessage
encryptedText: byte[]
encryptedAuthor: byte[]
encryptedSenderTimestamp: byte[]
encryptedImportantFlag: byte[]
signature: byte[]
verifyAndDecrypt(authorPublicKey: PublicKey, ourPrivateKey: PrivateKey): Message
dump(): void
delegate actions and subscribe for notifs
0..1
*
HumanIdentity
debug: boolean
username: String
keyPair: KeyPair
ourLocalRegistry: Registry
IDENTITY_BIND: String
watchAcks: ScheduledExecutorService
SAVE_PATH: Path
leave(): void
getUsername(): String
listParticipantsUsername(): List<String>
getParticipantPublicKey(username: String): Optional<PublicKey>
getRemoteIdentityFromUsername(recipient: String): Identity
createChat(recipient: String): Chat
getRemoteIdentityFromChat(chatID: UUID): Identity
approveChat(chatID: UUID): void
refuseChat(oldChatID: UUID): void
subscribeViewForChatEvent(view: View): void
sendMessage(chatID: UUID, text: String, important: boolean): void
closeChat(chatID: UUID): void
remoteAskForChat(author: String, chatID: UUID): void
remoteApproveBackChat(chatID: UUID): void
remoteRefuseChat(chatID: UUID): void
remoteSendBroadcastMessage(signedMessage: SignedMessage): void
remoteSendMessageInChat(chatID: UUID, safeMessage: SafeMessage): void
remoteCloseChat(chatID: UUID): void
getChats(): Map<UUID,Chat>
addToChats(uuid: UUID, chat: Chat): void
removeTOChats(uuid: UUID): void
findUuidByUsername(recipientUsername: String): UUID
resolveRmiHostname(): String
remoteAcknowledgeMessage(chatId: UUID, messageId: UUID): void
startWatchAcks(): void
autoDisconnect(chat: Chat): void
getKeyPair(): KeyPair
generateKeyPair(): KeyPair
stopWatchAcks(): void
saveChat(): void
loadChats(): void
broadcast(text: String): void
View
start(directoryHost: String): void
showChatRequest(from: String): void
showChatClose(from: String): void
showChatRefuse(from: String): void
showChatApprobation(from: String): void
showChatMessage(msg: Message): void
showBroadcastMessage(msg: Message): void
getIdentity(): HumanIdentity
1
start
App
PORT: int
main(args: String[]): void
create
0..1
*
1
*
stores and persists
remotely discuss with
Chat
serialVersionUID: long
uuid: UUID
approved: boolean
otherUsername: String
pendingAcks: Map<UUID,Instant>
ACK_TIMEOUT: Duration
getUuid(): UUID
getOtherUsername(): String
getApproved(): boolean
setApproved(t: boolean): void
insertNewReceivedMessage(msg: Message): void
insertNewMessage(text: String, author: String, important: boolean): Message
getMessages(): List<Message>
resetAcks(): void
registerPendingAck(messageId: UUID): void
receiveAck(messageId: UUID): void
hasTimedOutAck(): boolean
1
stores
*
Message
serialVersionUID: long
texte: String
senderTimestamp: long
author: String
isImportant: boolean
uuid: UUID
getTexte(): String
getSenderTimestamp(): Long
getAuthor(): String
getIsImportant(): Boolean
getUuid(): UUID
generateBytesToSign(args: byte[]): byte[]
ShellView
currentChat: UUID
currentChatRecipient: String
joinedNetwork: boolean
scanner: Scanner
directoryHost: String
join(username: String): void
showHelp(): void
createSwitchChat(recipient: String): void
listChats(): void
closeChat(recipient: String): void
approveChat(recipient: String): void
refuseChat(recipient: String): void
sendMessage(text: String, important: boolean): void
broadcast(text: String): void
listParticipants(): void
showPrompt(): void
waitWithShellPrompt(): void
parseCommand(line: String): void
exit(): void
formatTimestamp(millis: long): String
showMessage(msg: Message): void
start(directoryHost: String): void
showChatRequest(from: String): void
showChatClose(from: String): void
showChatApprobation(from: String): void
showChatRefuse(from: String): void
showChatMessage(msg: Message): void
showHistory(): void
getIdentity(): HumanIdentity
showBroadcastMessage(msg: Message): void
1
use as client
1
Directory
host: String
rsa: RSA
registeredUsername: String
list(): List<Entry>
join(username: String, keyPair: KeyPair): void
requestRegisterChallenge(username: String, publicKeyBase64: String): String
leave(keyPair: KeyPair): void
readStream(is: InputStream): String
removeUser(username: String, keyPair: KeyPair): void
openConnection(path: String, method: String): HttpURLConnection
writeBody(connection: HttpURLConnection, body: String): void
requestLeaveChallenge(username: String): String
buildRegisterSignature(signedContent: String, keyPair: KeyPair): String
buildLeaveSignature(signedContent: String, keyPair: KeyPair): String
signBase64(content: String, keyPair: KeyPair): String
requireKeyPair(keyPair: KeyPair): KeyPair
encodePublicKey(keyPair: KeyPair): String
requireUsername(username: String): String
httpFailure(action: String, status: int, connection: HttpURLConnection): RuntimeException
Entry
username: String
ip: String
publicKey: PublicKey
username(): String
publicKey(): PublicKey
ip(): String
return structured
connects via HTTP
DirectoryServer
LOGGER: Logger
rsa: RSA
directory: Map<String,Content>
secureRandom: SecureRandom
leaveChallenges: Map<String,LeaveChallenge>
registerChallenges: Map<String,RegisterChallenge>
CHALLENGE_TTL_MS: long
main(args: String[]): void
handleList(exchange: HttpExchange): void
handleRegisterChallenge(exchange: HttpExchange): void
handleRegisterConfirm(exchange: HttpExchange): void
handleRemoveChallenge(exchange: HttpExchange): void
handleRemoveConfirm(exchange: HttpExchange): void
readBody(is: InputStream): String
sendEmpty(exchange: HttpExchange, status: int, message: String): void
sendText(exchange: HttpExchange, status: int, text: String): void
sendText(exchange: HttpExchange, status: int, text: String, message: String): void
logOutcome(exchange: HttpExchange, status: int, message: String): void
EntryToStringLine(entry: Map.Entry<String,Content>): String
EntryToStringLine(username: String, ip: String, pubkey: String): String
formatEntryLine(username: String, ip: String, pubkey: String): String
IsUniqueIp(ip: String): boolean
generateChallenge(): String
SignedMessage
rsa: RSA
signature: byte[]
verifySignature(publicKey: PublicKey): boolean
getBytesToSign(): byte[]